

```

Windows PowerShell
[+] Running 9/9
=> [kali] exporting to image 3.2s
=> => exporting layers 0.0s
=> => exporting manifest sha256:16d06bbde88d71c10b9d7b1d0dab1771a6e22426e4e17400a467b4244aed4966 0.0s
=> => exporting config sha256:208188aec5149d600fd38d1f2cd26ae59f9d513f22d7a1e42931e4d591111a7b 0.0s
=> => naming to docker.io/library/workshop-kali:latest 0.0s
=> => unpacking to docker.io/library/workshop-kali:latest 3.2s
=> CACHED [snort3 2/2] RUN sed -i 's|http://deb.debian.org/debian|http://archive.debian.org/debian|g' 0.0s
=> [snort3] exporting to image 0.8s
=> => exporting layers 0.0s
=> => exporting manifest sha256:a65b0641929f98c0dd2f8248cc4c6b86aae7a51a4eef620a9ffbe84e18e0fa62 0.0s
=> => exporting config sha256:d6491c357ee103bb0b95e2aae1dee0d317cbdd42fbda9d0e2bd7f9f3c61df3d2 0.0s
=> => naming to docker.io/library/workshop-snort3:latest 0.0s
=> => unpacking to docker.io/library/workshop-snort3:latest 0.8s
=> [metasploitable] resolving provenance for metadata file 0.0s
=> [snort3] resolving provenance for metadata file 0.0s
=> [kali] resolving provenance for metadata file 0.0s
[+] Running 9/9
✓kali Built 0.0s
✓metasploitable Built 0.0s
✓snort3 Built 0.0s
✓Network workshop_pentest-network Created 0.1s
✓Container metasploitable Started 0.9s
✓Container snort3 Started 1.0s
✓Container dvwa_db Started 1.0s
✓Container kali Started 1.1s
✓Container dvwa Started 0.8s
PS C:\workshop>

```

Figure 1: Security testbed set up completion status

Metasploitable: This vulnerable virtual machine is designed for use with the Metasploit Framework. Metasploitable is an excellent environment for practicing penetration testing, particularly for network-based exploits. It's loaded with vulnerable services, making it a great resource for learning how exploits work in a real-world scenario.

Using Docker Compose, we will configure and deploy three containers to create a comprehensive and flexible security test bed. This setup will allow you to experiment with a variety of security tools and techniques in an isolated, controlled environment without the risk of harming your main system.

▪ **Step 1: Add images added in compose.yml**

The `compose.yml` file is the backbone of Docker Compose. It is a simple, human-readable file that defines all the services (containers) you want to run, including their configurations, such as port mappings, network settings, and volumes for persistent storage. For this security test bed, the `compose.yml` file will specify the images for Kali Linux,

DVWA, and Metasploitable, along with the necessary parameters to ensure they run correctly and are accessible from your host machine.

To make this process easier, you can either create the `compose.yml` file from scratch or download a pre-configured one that we've provided. This file ensures that the containers run in harmony, automatically configuring their interactions. It also makes it simple to start, stop, and scale your security environment with just a few commands.

Docker Compose is used because it:

- Simplifies multi-container deployment
- Enables easy configuration through YAML
- Allows services to run in isolated containers
- Supports networking between containers
- Has a one-command setup for complex environments

▪ **Step 2: Create a compose.yml file**

Rather than manually creating each container and configuring their settings, we could create a `compose.yml` file that defines all the containers required for

your security lab. This file includes the configuration for Kali Linux, DVWA, Snort3 and Metasploitable, and ensures that they work together seamlessly. It handles everything from downloading the necessary Docker images to setting up the required networking between the containers.

Let us create and save this file in an appropriate directory on your system where you want to run the security test bed.

What does the `compose.yml` file include?

- The images for Kali Linux, DVWA, Snort3 and Metasploitable.
- Port mappings to allow access to services from your host machine (e.g., DVWA web interface).
- Settings to ensure that each container runs correctly and can communicate with the others.

This step reduces the need for manual setup, saving time and ensuring that everything is configured correctly from the start.

▪ **Step 3: Run Docker Compose**

Once you have the `compose.yml` file in place, it's time to bring your